SC2002 Group Assignment

Module Code: SC2002
Group number: Group 5
Tutorial Number: FDAB
Member names: Rajput Shagun (U2523117A)
Ng Yan Quan Ryan (U2521838E)
Pok Zheng Yong (U2521197A)
Raj Kumar Raghul (U2522765K)
Moon Seongwon (U2220665H)
Date of submission: 19/04/2026

Table of contents

Table of contents 2
1. Introduction 3
2. System Overview 4
3. Additional Features and Functionalities 5
4. Design Considerations 6
5. Implementation Notes 10
6. Reflection 10
7. Conclusion 11
8. UML Class Diagram 12
9. UML Sequence Diagram 13
10. GitHub Repository Link 14
11. Declaration of Original Work 15
12. Declaration on Use of GAI 17
    12.1. Rajput Shagun 17
    12.2. Ng Yan Quan Ryan 18
    12.3. Pok Zheng Yong 19
    12.4. Raj Kumar Raghul 20
    12.5. Moon Seongwon 21

1. Introduction

This report documents the design and implementation of a turn-based combat arena built for our SC2002 project. The game places one of two playable heroes, a Warrior or a Wizard, against waves of enemies across three difficulty tiers. The game then lets the player choose between two actions per turn together with a consumable inventory that includes potions, smoke bombs and power stones. The brief specified the game rules, combat flow, status effects and item behaviour. Our job was the design work of the combat arena.

This report covers our design considerations, UMl, the patterns we applied and the problems that we ran into because the decisions in one area affects the others. The report covers the system, the additional features that go beyond the minimum spec, the design principles and patterns we applied and the difficulties we met along the way.

The class and sequence diagrams referenced throughout sit alongside this report as separate artefacts. We refer to them here to keep the flow readable.

2. System Overview

The program is split into four main layers that correspond to responsibility rather than package. The entry point is Main which constructs a GameManager and prompts it to run. GameManager owns the top level loop which greets the player, asks for the difficulty and character choice, spawns enemies through LevelManager and hands control to BattleEngine for the actual fight. Once a battle is over, GameManager asks if the player wants another run.

BattleEngine is the rules engine. It holds the live list of combatants, advances rounds, ticks status effects at round start and decides when a battle is over. We designed it such that it intentionally does not have any information on the user interface. The only information it returns after a turn is a GameState snapshot, which is an immutable Data Transfer Object (DTO) which carries CombatantSnapshot records for every fighter on the board, round number and a victory/defeat/ongoing flag.

CLIHandler is the presentation layer. It reads from System.in, validates inputs against the available options and writes formatted output. It is the only class that calls System.out directly. The game manager passes it a GameState each time the display needs refreshing, so the UI cannot accidentally reach back and mutate the live combatants. This separation proved useful when a combat display bug appeared as fixing it was contained to GameManager and CLIHandler and the rules engine stayed untouched.

The domain layer contains Combatant, its subclasses (Warrior, Wizard, Goblin, Wolf), the Action hierarchy (BasicAttack, Defend, ShieldBash, ArcaneBlast), the Item hierarchy (Potion,SmokeBomb, PowerStone) and the StatusEffect hierarchy (DefenseBoost, AttackBuff, StunEffect, DamageZeroEffect). TurnOrder which is implemented by SpeedOrder and EnemyAction which is implemented by BasicEnemyStrategy are injected into BattleEngine so the combat rules can vary without rewriting the engine.

A typical round runs in a sequence as follows. BattleEngine.startRound increments the round counter, ticks every combatant's active effects once (handling stuns, defence buffs and attack buffs uniformly) and checks for end-of-battle conditions. GameManager then walks through the turn order returned by SpeedOrder, asks the CLI for the player's choice, constructs the right Action/Item and calls BattleEngine.executeTurn. After every turn, the CLI renders the refreshed GameState so the player sees HP changes, new status effects and the flow of events immediately.

3. Additional Features and Functionalities

3.1 Unique enemy labelling in mixed battles

When the player meets two enemies of the same type in a single battle, the CLI adds an A/B label so the player can tell them apart. The labels persist across rounds, which matters when targeting prompts appear (e.g.: the player sees "Wolf A (HP:20/50)" and "Wolf B (HP: 50/50)" rather than two identical "Wolf" entries. The logic is entirely in CLIHandler and the domain classes know nothing about display names so this labelling rule stays where it belongs.

3.2 Replay Loop

The game prompts the player after each battle resolves, reinitialising the manager cleanly so the player can try a different character or difficulty without restarting the JVM. The loop is confined to GameManager.run such that BattleEngine is constructed afresh each time, which ensures edge cases around lingering state from the previous battle are avoided.

3.3 Robust Input Validation

All user input flows through CLIHandler's validated read methods. This handles wrong inputs which includes non-numeric input, out-of-range selections and blanks and re-prompts with a clear message. This keeps the game loop resilient during testing such that we do not have to guard against wrong inputs inside GameManager, because the issue has been resolved in CLI.

3.4 Immutable GameState DTO

BattleEngine builds a GameState containing CombatantSnapshot records. Each snapshot copies the combatant's name, HP, stats, status-effect name and alive flag. The copy happens in the snapshot's constructor such that the UI can sort or format its contents without any risk to the rules engine.

3.5 Structured combat display

The CLI prints a consistent "before / after" block for every significant action, listing HP changes, new status effects and any lingering conditions. There is a method for each display scenario (basic attack, shield bash, arcane blast, defend, item use) which made it straightforward to refine when we discovered that ArcaneBlast's per-target ATK needed to be computed before kill buffs and not after.

4. Design Considerations

4.1 Abstractions

The main abstractions are Combatant, Action, Item and StatusEffect. Each of these abstractions captures a family of behaviours that vary in details but share a common protocol, which is exactly what an abstract class or interface is for. Combatant is an abstract class because all fighters share real state (currentHP, maxHP, attack, defense, speed, activeEffects) and real behaviour (takeDamage, isAlive, updateStatusEffects). The subclasses Warrior, Wizard, Goblin and Wolf only differ in their starting stats. For heroes, they differ in their item inventories. Using an abstract class rather than an interface lets us share the state and a default takeDamage implementation in one place. Action is an interface with a single execute method. The four actions (BasicAttack, Defend, ShieldBash, ArcaneBlast) have almost nothing in common other than them being a combatant's action during its turn, so a shared base class would have forced unnecessary state on classes that don't need it. The interface is deliberately minimal so the engine does not need to know which action it is running.

Item is also an interface as Potion, SmokeBomb and PowerStone implement use. Items carry their own charges or effects and PowerStone in particular wraps another action instead of duplicating its logic. StatusEffect is the abstraction we revised most during development. Early drafts used a flat data class with an effect-type enum, but we ran into awkward branching every time a new effect behaviour was needed. We refactored an abstract class with apply(Combatant), decrement(), isExpired() and remove(Combatant) methods, so each effect has its own lifecycle. DefenseBoost, AttackBuff, StunEffect and DamageZeroEffect override only the methods they need.

4.2 SOLID Principles

4.2.1 Single Responsibility (SRP)

The clearest application of Single Responsibility is the split between BattleEngine and CLIHandler. The engine handles rules while the CLI handles display and input. We broke this rule early on, when GameManager held a reference to the combatants and printed directly. The result was that any UI tweak ran the risk of breaking combat and any combat tweak could corrupt the display. Separating the two, with GameState as the bridge between them made both sides simpler. GameManager itself is arguably the class that least applies Single Responsibility. It orchestrates game flow, wires up dependencies, translates CLI choices to Actions and formats some of the post-action displays. We considered splitting it into a pure controller but decided the class should be kept as the main orchestrator.

4.2.2 Open-Closed (OCP)

Adding a new status effect is an open-closed win. For example, to introduce a "Poison" effect that damages the target each round, we would create a new subclass of StatusEffect overriding decrement to apply damage and nothing in BattleEngine, GameManager or CLIHandler would need to be changed. The same can be said for new Actions and new Items. We just have to construct the class, register it in the appropriate menu and the rest of the system picks it up because it programs to the interface. The one place we fell short is Combatant.getDefense, which contains an instanceof DefenseBoost check to fold the buff into the returned value. We tried routing it entirely through the status-effect system, which

would have made getDefense oblivious to specific effect types, but the read-time semantics made the instanceof version the more readable option. We flag this as a deliberate trade-off rather than a clean Open-Closed adherence.

4.2.3 Liskov Substitution (LSP)

Every subclass of Combatant is substitutable. Warrior and Wizard differ in starting stats and item inventory, but both respect the Combatant contract for HP, attack, defence, and effect-tracking. The same holds for the Action and Item hierarchies in which no implementation throws UnsupportedOperationException and no implementation narrows preconditions. The test was whether BattleEngine drove a turn without knowing the concrete type, in which it does. The switch on action type happens in GameManager where we are already translating a menu choice into an object, not inside the engine.

4.2.4 Interface Segregation (ISP)

Our interfaces are relatively small. Action, Item, EnemyAction and TurnOrder have one method. BattleObserver has three methods (onTurnStarted, onActionResolved, onBattleEnded) but any implementation that cares about only one event can no-op the others and the three correspond to genuinely distinct lifecycle points.

We considered merging EnemyAction into Action to avoid two action types but decided that enemies' decision-making is a different concern from action execution. An EnemyAction produces an Action given the current board in which the Action itself carries out the decision. Collapsing them would have forced every enemy decision maker to know how to execute its choice, which is exactly the kind of fat interface ISP warns against.

4.2.5 Dependency Inversion (DIP)

BattleEngine accepts TurnOrder and EnemyAction through its constructor. The engine depends on the interfaces, not on SpeedOrder or BasicEnemyStrategy. We swapped in a simpler turn order for a scripted replay of the spec's worked examples without touching the engine. We also route everything through GameState rather than letting the UI grab the combatant list. This is dependency inversion in spirit if not in name: the UI depends on an abstraction that both sides agree on instead of the concrete Combatant class whose methods could drift.

4.3 Design Patterns

4.3.1 Strategy

SpeedOrder implements TurnOrder and BasicEnemyStrategy implements EnemyAction. The engine calls the injected strategy. If we ever wanted a "fastest-first with tiebreakers" ordering or a smarter enemy AI that chooses between attack and defend, we would write a new strategy and pass it in.

4.3.2 Observer

BattleEngine maintains a list of BattleObserver listeners and calls notifyObservers after every significant lifecycle event. The current observer implementations are small, but the pattern is in place and the engine is decoupled from whatever the observer might want to watch a battle unfold.

4.3.3 Template Method

Combatant.takeDamage is a minor template method: the base class handles the zero damage short circuit, the clamp to zero and the update to currentHP. Subclasses do not override it now but could override hooks such as modifyIncomingDamage without rewriting the clamping logic. We kept the hook surface minimal because a full template with a dozen protected overrides would be too much for a four-subclass hierarchy.

4.3.4 Data Transfer Object

GameState and CombatantSnapshot are classic DTO. They are not the Memento pattern, but have a similar intention of freezing a moment in time so another part of the system can inspect it without risk of mutation.

4.3.3 Composition over inheritance for items

PowerStone demonstrates a clear composition over inheritance. Rather than subclassing ShieldBash or ArcaneBlast, PowerStone wraps a reference to the Action associated with the player's class and delegates to it when used. This was the cleanest way to let the character cast their signature move for free without forcing the item hierarchy and the action hierarchy to become a grid of subclasses.

4.4 Coupling and Cohesion

We judged coupling class by class, and the breakdown looks something like this. BattleEngine has low afferent coupling as only GameManager and the observers depend on it and also moderate efferent coupling because it touches Combatant, Action, TurnOrder, EnemyAction and BattleObserver. That different coupling is almost entirely to interfaces and an abstract class, which keeps the engine itself easy to modify. GameManager has the highest coupling in the project. It wires BattleObserver, CLIHandler, BattleEngine, LevelManager and the concrete hero and enemy classes together. We are fine with this because GameManager is explicitly the composition root. Concentrating the wiring there keeps everything else free to depend only on interfaces. CLIHandler has a few instanceof checks in its end-of-round display. These are a source of coupling we considered removing with a visitor pattern or a getDisplayInfo method on Combatant, but both alternatives would have pushed presentation concerns down into the domain model. We preferred the coupling to sit in the one class whose role it already is to know about display.

Cohesion instead is generally high. Each Action class does one thing, each StatusEffect class manages its own lifecycle and lastly each Item class knows its own use. The one class with mixed cohesion is GameManager because of its orchestrator role. We accept this trade-off on the same grounds as above.

4.5 Trade-offs and Alternatives Considered

Firstly, status effects tick timing. Status effects could tick at round start, at the start of each combatant's turn or at the end of each turn. We tried all three. Ticking per-turn caused stun and defence buffs to expire a round too early, because a combatant that had just been stunned would lose the effect before its own turn processed. Ticking at round end created inconsistencies when a combatant died mid-round and the ticker tried to decrement effects on a corpse. Ticking once at round start, before turns run was the least surprising option and we documented the choice in BattleEngine with a code comment such that we do not re-introduce the earlier bugs.

Secondly, ArcaneBlast damage computation. ArcaneBlast hits every enemy and applies an AttackBuff to the caster for each kill it produces mid-loop. This means the caster's attack can rise between the first and last target in a single cast. Our original display code read the caster's ATK after execution and used it for every target, which produced the wrong per-target numbers once a kill buff had stacked. We fixed this by writing a private simulateArcaneBlast method in GameManager that mirrors the engine's loop without mutating any state, records the per-target damage and is used purely for the summary. The engine's execution remains authoritative for state changes. An alternative was changing the Action interface to return a result object, but that would have forced every Action implementation and every caller to carry a result type that only ArcaneBlast needs.

Thirdly, the instanceof check in Combatant.getDefense. We could have implemented defence buffs as a generic stat modifier that every effect class could contribute to, making getDefense oblivious to specific effect types. We chose the instanceof form because the alternative would have required a wider StatusEffect contract (a getDefenseBonus method on every effect, returning zero for most), which is a worse tradeoff, which is a wider interface for marginal gain in modularity. We flag this in the code with a comment.

Next, immutable snapshot versus live reference. Snapshotting every combatant every round has a runtime cost. We measured it as negligible for the number of combatants the game ever has, which is 4 and the safety benefit is that the UI physically cannot corrupt combat state. The calculus might change for a larger battle, at which point a read-only interface wrapping the live objects would be a reasonable refactor.

Lastly, enemy AI complexity. BasicEnemyStrategy always chooses BasicAttack. We considered a weighted random choice between attacking and eventually defending, but the spec's worked examples assume deterministic enemy behaviour and anything more elaborate would have broken the comparison. The strategy interface is in place so a Hard difficulty could ship a smarter enemy AI later without touching the engine.

5. Implementation Notes

Dead combatants are pruned from the combatant list twice per round: once at the top of startRound before effects tick, so an effect cannot act on a dead body and once inside executeTurn (such that a combatant that kills an enemy and then has another effect tick does not see it). The double prune is defensive but cheap. Cooldowns are tracked separately from status effects, in a CooldownTracker attached to each combatant. We initially tried to represent cooldowns as a StatusEffect, but cooldowns are the combatant's own constraint on an action it owns, while status effects are things that have been applied to the combatant from outside. Forcing them into the same abstraction confused both, so we kept them separate.

6. Reflection

6.1 Difficulties Encountered

The biggest difficulty was getting ArcaneBlast's per-target damage right when the caster's attack mutates mid-cast. The bug did not show up in the Warrior's worked example, because shield bash and basic attack do not buff the attacker. It showed up when we traced the spec's Medium Wizard example by hand, compared the expected output to our actual output and found the Wolves' reported damage did not match. The solution was to simulate the blast in display code without touching live state. The issue was obvious once we understood the cause, but it took a careful side-by-side trace to see the cause in the first place. The lesson is that worked examples from the spec are not there only to check correctness but also to check our mental model matches the code's actual behaviour.

A second difficulty was deciding where rendering logic should live. Our first cut had CLIHandler reaching directly into the combatant list, which made every bug we hit ambiguous. Rewriting around GameState was a chunk of work but removed the ambiguity completely.

A third difficulty was deciding what features to add. The assignment permits many possible features and it always feels better to add more. We cut a planned critical hit mechanic and a status effect icons display because they would have expanded the surface area of the code without really improving the design.

6.2 Key Learnings

We came to a realisation, that the smaller an interface, the easier it was to add something new without breaking existing code.Every time we reached for an abstract class with more than three or four methods, we ended up with subclasses that had to override half of the methods just to leave them empty. The Action and Item interfaces, which are each a single method, are the easiest parts of the code to extend.

The second lesson was that immutable snapshots are worth it. We spent a shorter time converting GameState to a true Data Transfer Object (DTO), and the time saved on debugging UI-state confusion paid that back several times over.

The third lesson is more indirect. We think it is beneficial to be honest with the flaws of our code. Combatant.getDefense is not a perfect OCP example, GameManager is not a perfect SRP example and CLIHandler has a little instanceof logic we would prefer to remove. Pretending otherwise in this report would be cheaper than the work of fixing it, but would not help us to learn. Acknowledging the trade offs is part of the design work.

6.3 Possible Improvements

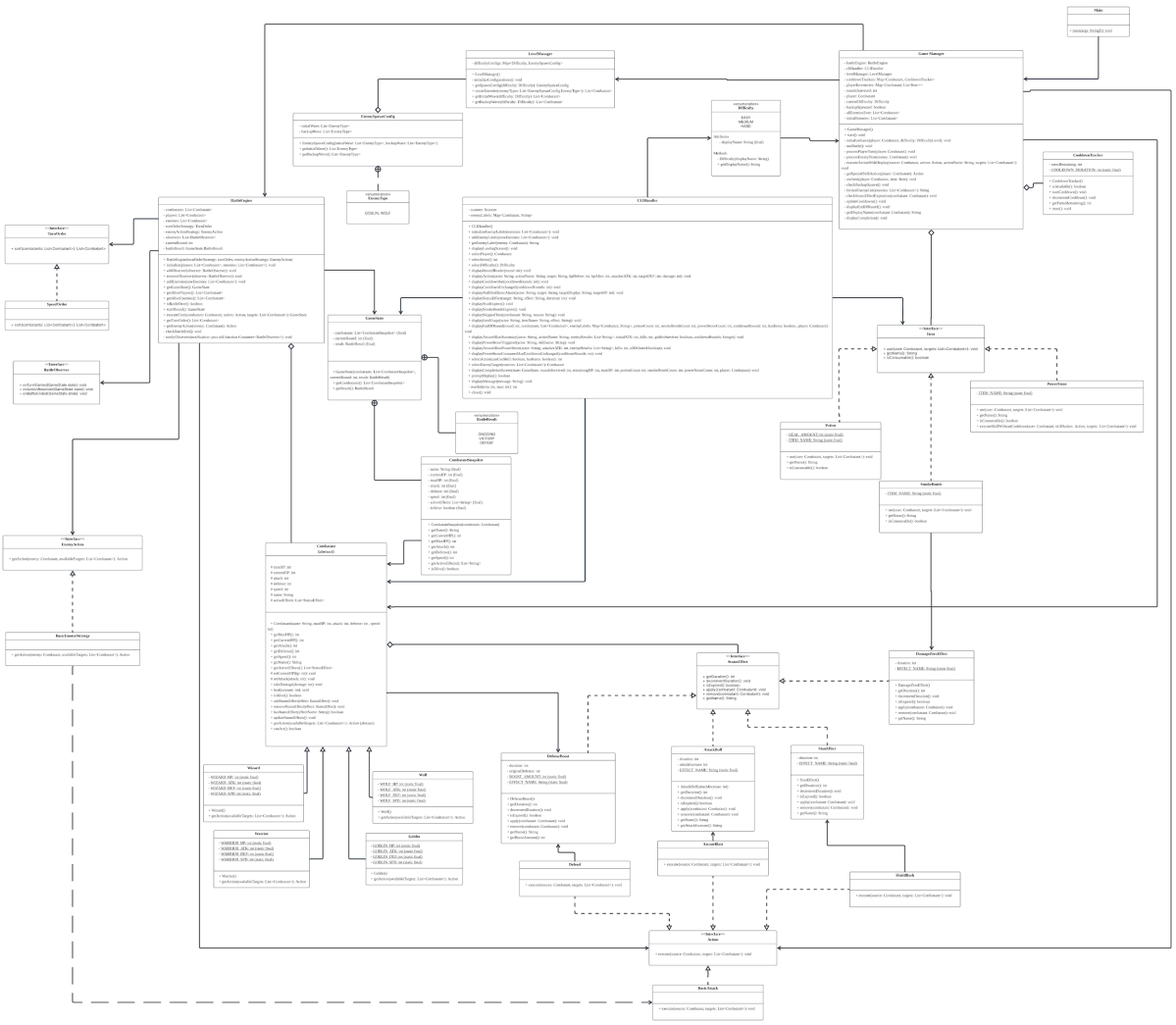
Given more time, the changes we would pursue

1. Add a getDisplayData method on Combatant that returns a small record of what the CLI needs such that the instanceof ladder in end-of-round rendering can be replaced with a polymorphic call
2. Generalise StatusEffect so defence and attack bonuses are contributed through the effect class itself, removing the instanceof branch in Combatant.getDefense
3. We'd also move the cooldown turns remaining into CombatantSnapshot, as right now the display has to reach back into the live tracker which defeats the point of having a snapshot
4. Introduce a proper logger behind BattleObserver so a replay of any battle can be reconstructed from an event stream
5. Extend the enemy AI strategy with a second, smarter implementation to justify the abstraction

7. Conclusion

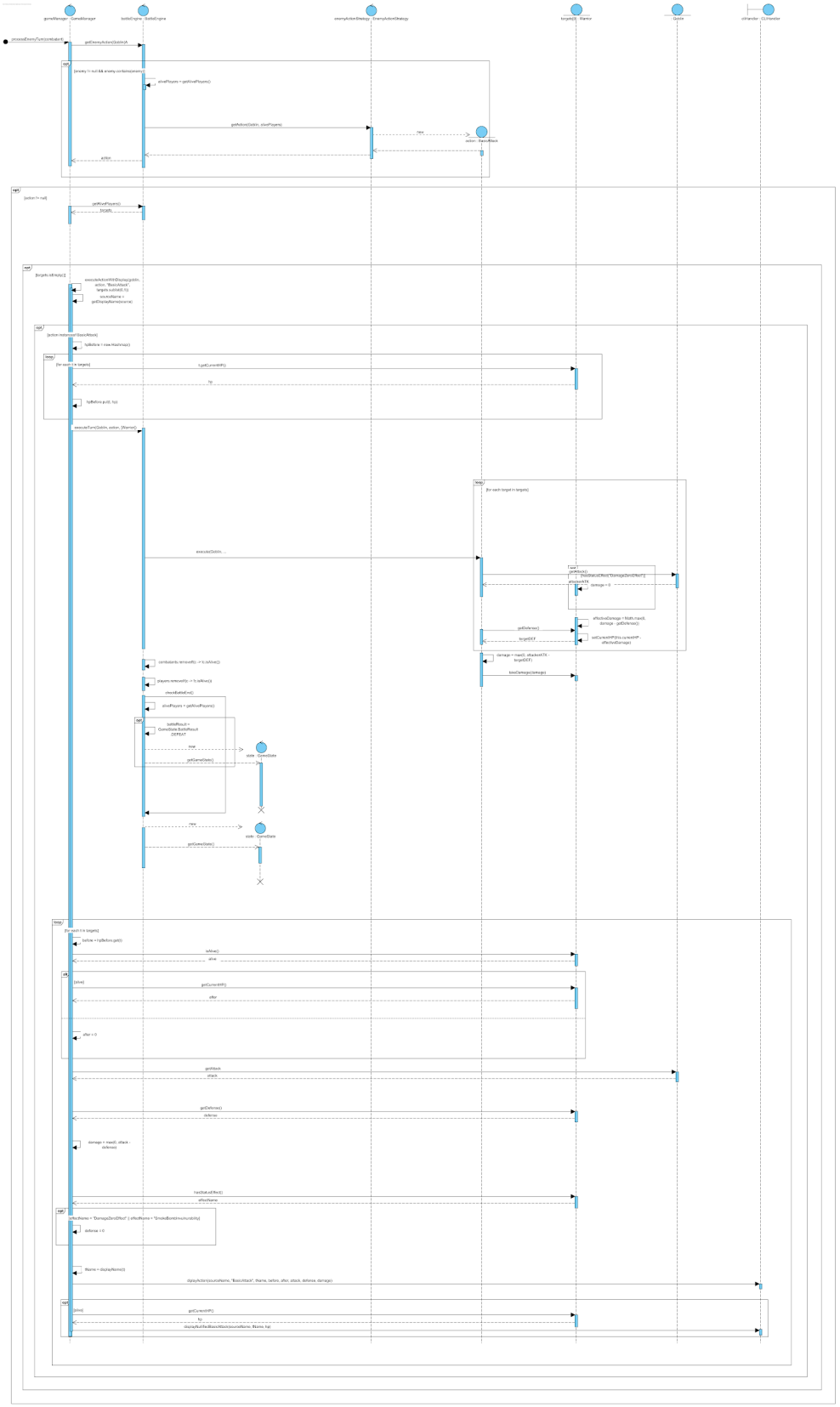
The turn-based combat arena is built around a small number of abstractions (Combatant, Action, Item, StatusEffect and GameState) held together by a rules engine that knows nothing about the UI and a UI that cannot mutate the rules engine. The SOLID principles and the Strategy, Observer, Template-Method and DTO patterns shaped those abstractions and the trade offs we made are recorded alongside the places we held the line. The development was iterative, the hardest bug was caught by tracing worked examples against the code by hand and the final system faithfully implements the spec while leaving room to grow.

8. UML Class Diagram



9. UML Sequence Diagram

(Enemy's turn: Goblin executes basic attack on Warrior)



10. GitHub Repository Link

https://github.com/Moon0002/SC2002_Assignment_battlearena

11. Declaration of Original Work

Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Signature/Date
Ng Yan Quan Ryan (U2521838E)	SC2002	FDAB	19 April 2026
Pok Zheng Yong (U2521197A)	SC2002	FDAB	19 April 2026
Moon Seongwon (U2220665H)	SC2002	FDAB	19 April 2026
Raghul Rajkumar (U2522765K)	SC2002	FDAB	19 April 2026

Shagun Rajput (U2523117A)	SC2002	FDAB	 19 April 2026

12. Declaration on Use of GAI

12.1. Rajput Shagun

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects) I, Shagun Rajput, shagun001@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design & Programming
2. Course Code: SC2002
3. Instructor: SRITHARAN NIRUTHIKKA
4. Title of Assignment/Project Submission: SC2002 Turn-based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):
i. Used GAI as permitted to assist in generating key ideas only. ☐
ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.☑

I also declare that I have :
a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work

Shagun Rajput
Student Name

_____19 April 2026
Signature / Date

12.2. Ng Yan Quan Ryan

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects) I, Ng Yan Quan Ryan (student name), yng151@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design & Programming
2. Course Code: SC2002
3. Instructor: SRITHARAN NIRUTHIKKA
4. Title of Assignment/Project Submission: SC2002 Turn-based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):
i. Used GAI as permitted to assist in generating key ideas only. ☐
ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.☑

I also declare that I have :
a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work

Ng Yan Quan Ryan
Student Name

19 April 2026

Signature / Date

12.3. Pok Zheng Yong

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects) I, Pok Zheng Yong (student name), ZPOK001@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design & Programming
2. Course Code: SC2002
3. Instructor: SRITHARAN NIRUTHIKKA
4. Title of Assignment/Project Submission: SC2002 Turn-based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):
i. Used GAI as permitted to assist in generating key ideas only. ☐
ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.☑

I also declare that I have :
a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work

_Pok Zheng Yong____
Student Name

_____19 April 2026_
Signature / Date

12.4. Raj Kumar Raghul

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects) I, Rajkumar Raghul (student name) ,raghul001@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design & Programming
2. Course Code: SC2002
3. Instructor: SRITHARAN NIRUTHIKKA
4. Title of Assignment/Project Submission: SC2002 Turn-based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):
i. Used GAI as permitted to assist in generating key ideas only. ☐
ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.☑

I also declare that I have :
a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work

Raj kumar Raghul
Student Name

Signature / Date

12.5. Moon Seongwon

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects) I, Moon Seongwon, moon0002@e.ntu.edu.sg honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design & Programming
2. Course Code: SC2002
3. Instructor: SRITHARAN NIRUTHIKKA
4. Title of Assignment/Project Submission: SC2002 Turn-based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):
i. Used GAI as permitted to assist in generating key ideas only. V
ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.☐

I also declare that I have :
a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work

Moon Seongwon
Student Name

4/19/2026

Signature / Date